RAG Fundamentals

A practical guide to retrieval-augmented generation and common AI Engineering interview topics

Purpose: This original sample document is designed for document loading, chunking, embedding, retrieval, and question-answering experiments. Its headings and paragraphs provide useful structure for metadata-aware LangChain ingestion.

1. What is RAG?

Retrieval-augmented generation, or RAG, is an application pattern that supplies a language model with information retrieved from an external knowledge source at query time. The model does not rely only on knowledge encoded during training. It receives selected evidence in its prompt and uses that evidence to produce a grounded answer.

A basic RAG pipeline has two paths. The indexing path loads source documents, extracts text and metadata, divides content into chunks, creates embeddings, and stores searchable representations. The query path transforms a user question into a search request, retrieves relevant chunks, optionally reranks them, constructs a prompt, calls a model, and returns an answer with evidence.

2. Core components

Document loading: A loader converts files, web pages, APIs, or database rows into a consistent document representation. Good loaders preserve provenance such as source URL, file name, page number, publication date, and document identifier.

Chunking: Chunking divides content into retrieval units. Fixed-size chunks are simple, while recursive, semantic, or structure-aware chunking can better preserve meaning. Chunk size and overlap are empirical choices: large chunks carry more context but may dilute relevance; small chunks are precise but may lose surrounding facts.

Embeddings: An embedding model maps text into numeric vectors whose distance approximates semantic similarity. The same compatible model and preprocessing rules must be used for indexed content and queries. Changing the embedding model usually requires reindexing.

Retrieval: Dense retrieval searches embeddings. Sparse retrieval, such as BM25, matches lexical signals. Hybrid search combines both and is often stronger when exact names, citations, identifiers, or specialized terminology matter. Metadata filters reduce the search space using attributes such as country, date, source, or category.

Reranking: A reranker scores a small candidate set with a more expensive model. This separates broad candidate recall from precise final ordering. It can improve relevance but adds latency and cost.

Generation: The prompt should define the task, include retrieved context, instruct the model to distinguish evidence from inference, and specify behavior when evidence is insufficient. Citations should be linked to stable source metadata rather than invented by the language model.

3. Retrieval quality and evaluation

RAG evaluation should separate retrieval from generation. Retrieval metrics include recall at k, precision at k, mean reciprocal rank, and normalized discounted cumulative gain. Generation evaluation can measure faithfulness, answer relevance, completeness, citation correctness, and refusal quality when the corpus lacks an answer.

A useful evaluation set contains representative questions, expected evidence, and grading criteria. Production traces can reveal new failure cases, but private data must be handled according to its retention and access policies. Automated model-based grading is helpful, but it should be calibrated against human judgments.

4. Common failure modes

Frequent problems include poor text extraction, missing metadata, chunks that split important sections, embedding mismatch, irrelevant top-k results, duplicated documents, outdated indexes, context-window overflow, prompt injection inside retrieved text, and answers that overstate what the evidence supports.

Debug the pipeline stage by stage. First inspect extracted text. Then verify chunks and metadata. Next test whether the expected chunk appears in retrieval results. Finally inspect the exact context sent to the model and compare the answer with that context. End-to-end accuracy alone does not identify which stage failed.

5. Production AI Engineering topics

Data lifecycle: Production systems need repeatable ingestion, idempotent updates, deduplication, versioning, deletion, and lineage. A source document, extracted representation, chunks, embeddings, and generated outputs should remain traceable to one another.

Reliability: External models and stores can fail or throttle. Use timeouts, bounded retries with backoff, concurrency limits, dead-letter handling where appropriate, and idempotency for repeated jobs. Cache only when freshness and access boundaries are understood.

Observability: Track ingestion throughput, extraction failures, index freshness, retrieval latency, model latency, token usage, cost, empty retrievals, citation coverage, and evaluation scores. Trace individual requests across retrieval and generation while avoiding unnecessary exposure of sensitive content.

Security: Treat retrieved documents as untrusted input. Enforce authorization before retrieval, isolate tenants where applicable, protect secrets, minimize stored personal data, and defend against prompt injection and data exfiltration. The model must not be the component responsible for access control.

Cost and latency: Important controls include model selection, chunk count, prompt length, embedding batches, reranking depth, caching, streaming, and asynchronous processing. Optimize after measuring a real workload and preserve quality guardrails.

6. Common interview questions

Why use RAG instead of fine-tuning?

RAG is appropriate when answers depend on changing or private facts that must be retrieved and cited. Fine-tuning is usually better for changing model behavior, style, or task performance. They solve different problems and can be combined.

How would you choose a chunking strategy?

Start from document structure and expected questions. Preserve semantically complete units, attach useful metadata, and test several sizes and overlaps on an evaluation set. Choose using retrieval and answer quality together with latency and cost.

What is the difference between dense and sparse retrieval?

Dense retrieval uses vector similarity and captures paraphrases and semantic relationships. Sparse retrieval uses token-level signals and is strong for exact terminology. Hybrid retrieval combines their strengths.

How do you reduce hallucinations in a RAG system?

Improve retrieval, require evidence-grounded answers, preserve clear source boundaries, instruct the model to abstain when evidence is insufficient, validate citations, and evaluate faithfulness. RAG reduces some hallucinations but does not eliminate them.

How do you detect stale or duplicate content?

Use stable source identifiers when available, normalized URLs, content hashes, domain-specific identifiers, and version metadata. Keep ingestion idempotent and define whether a change creates a new

version or replaces an existing representation.

What would you monitor in production?

Monitor data freshness, pipeline failures, retrieval quality, empty results, latency by stage, model errors, token consumption, cost, citation correctness, user feedback, and security events. Use traces to connect a generated answer to its query and retrieved evidence.

How would you test a RAG application?

Use unit tests for deterministic transformations, integration tests for loaders and stores, retrieval evaluations with labeled evidence, generation evaluations for groundedness and usefulness, adversarial security tests, and end-to-end tests for representative workflows.

7. Practical checklist

Before launch, verify source permissions and provenance; inspect extracted text; define chunking and metadata rules; establish update and deletion behavior; build an evaluation set; measure retrieval before generation; require traceable citations; test insufficient-evidence behavior; protect access boundaries; and monitor freshness, quality, latency, and cost.